A - SEARCH

这题要求打印最终的所有可能出现的位置，注意到 ***r * c<2500*** ，是一个比较小的数据范围，考虑模拟。

注意到在每个时间点汇聚到同一点的所有状态完全等效，那么存储某点在某一时间是否可达，然后用BFS模拟即可

每次移动时间复杂度是 ***O(r * c)*** ，移动次数 ***N < 1000*** , 整体复杂度 ***O(r * c * N)***

代码：

```
#include<bits/stdc++.h>
using namespace std;
int mv[4][2]={
    {-1,0},{0,1},{1,0},{0,-1}
};
signed main(){
    ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);
    int r,c;cin>>r>>c;
    vector<vector<bool>>stop(r,vector<bool>(c));
    vector<vector<bool>>bfs(r,vector<bool>(c));
    {
        char temp;
        for(int i=0;i<r;i++){
            for(int j=0;j<c;j++){
                cin>>temp;
                if(temp=='X'){
                    stop[i][j]=true;
                }else if(temp=='*'){
                    bfs[i][j]=true;
                }
            }
        }
    }
    int n;cin>>n;
    int com=0;
    for(int q=0;q<n;q++){
        string command;cin>>command;
        if(command=="NORTH"){com=0;}
        else if(command=="EAST"){com=1;}
        else if(command=="SOUTH"){com=2;}
        else {com=3;}
        vector<vector<bool>>temp(r,vector<bool>(c));
        for(int i=0;i<r;i++){
            for(int j=0;j<c;j++){
                if(bfs[i][j]){
                    int x=i+mv[com][0],y=j+mv[com][1];
                    while(x>=0&&x<r&&y>=0&&y<c){
                        if(stop[x][y]||temp[x][y]){
                            break;
                        }else{
                            temp[x][y]=true;
                            x+=mv[com][0];y+=mv[com][1];
                        }
                    }
                }
            }
        }
        bfs=move(temp);
    }
    for(int i=0;i<r;i++){
        for(int j=0;j<c;j++){
            if(stop[i][j]){cout<<'X';}
            else if(bfs[i][j]){cout<<'*';}
            else{cout<<'.';}
        }
```

```
        cout<<'\n';
    }
}
```

B - 胜利大逃亡(续)

本题和**网格上 bfs 模版题**的区别是多了一个**用钥匙开门**的过程。如果你还没有掌握如何在网格上进行 bfs，请点击这里。

如何处理钥匙呢。不妨假设我们有一个**钥匙串**，上面有 A 到 J 十个位置，起始时钥匙串为**空**。当我们走到某一位置获得某把钥匙时，就往钥匙串上加入这把钥匙。所以说，**除了当前的横纵坐标以外，钥匙串的状态也是当前状态中的一维**。

而这个过程可以用**状态压缩**完成。具体来说，我们可以用一个长度为10的二进制数来表示当前钥匙获得情况。例如，0000000001 表示钥匙串中仅含钥匙 A，0000000101 表示含钥匙 A 与钥匙 C。这样每一个时刻我们拥有钥匙集合的状态都可以用一个**整型**表示。携带着钥匙串的状态，我们就可以判断在经过每一扇门时是否可以通过。

值得注意的一点是 vis 数组同时需要多开一个维度。很好理解，当我们携带不同钥匙串通过同一个点时应该视为两种情形。

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
const int N=25;
int n, m, t, sx, sy, ex, ey, v[N][N][1<<10];
char a[N][N];
int dx[4]={1, -1, 0, 0};
int dy[4]={0, 0, 1, -1};
struct node {
    int x, y, key, step;
};
bool out(int x, int y) {
    if(x<1||x>n||y<1||y>m) return 1;
    else return 0;
}
int bfs() {
    queue<node> q;
    q.push({sx, sy, 0, 0}); v[sx][sy][0]=1;
    while(!q.empty()) {
        auto [x, y, key, step]=q.front(); q.pop();
        for(int i=0; i<4; i++) {
            int tx=x+dx[i], ty=y+dy[i];
            if(v[tx][ty][key]||out(tx, ty)||a[tx][ty]=='*') continue;
            //点
            if(a[tx][ty]=='.'||a[tx][ty]=='@') {
                v[tx][ty][key]=1;
                q.push({tx, ty, key, step+1});
            }
            //钥匙
            if(a[tx][ty]>='a'&&a[tx][ty]<='j') {
                int nkey=key|(1<<(a[tx][ty]-'a'));
                v[tx][ty][nkey]=1;
                q.push({tx, ty, nkey, step+1});
            }
            //门
            if(a[tx][ty]>='A'&&a[tx][ty]<='J') {
                if((key>>(a[tx][ty]-'A'))&1) {
                    v[tx][ty][key]=1;
                    q.push({tx, ty, key, step+1});
                }
            }
            //终点
            if(a[tx][ty]=='^') {
                return step+1;
            }
        }
    }
```

```
    return LLONG_MAX;
}
void solve() {
    memset(v, 0, sizeof(v));
    for(int i=1; i<=n; i++) {
        for(int j=1; j<=m; j++) {
            cin>>a[i][j];
            if(a[i][j]=='@') sx=i, sy=j;
            if(a[i][j]=='^') ex=i, ey=j;
        }
    }
    int ans=bfs();
    if(ans<t) cout<<ans<<'\n';
    else cout<<-1<<'\n';
}
signed main() {
    ios::sync_with_stdio(0); cin.tie(0);
    while(cin>>n>>m>>t) solve();
    return 0;
}
```

C - Prime Ring Problem

题意

一个环由 n 个圆圈组成。将自然数 $1, 2, \ldots, n$ 分别填入每个圆圈中，且相邻两个圆圈中的数字之和必须为素数，第一个圆圈中的数字必须始终为 1。

按字典序输出所有符合条件的排列。

思路

数据范围很小，暴力搜索即可，从小到大枚举当前位置填的数字，搜索到 n 后特判最后一个数字和 1 的和是否为素数。

素数怎么判断都行，可以分解质因数，也可以用筛法。

code

```
#include <bits/stdc++.h>
using namespace std;

#define f(i,a,b) for(int i=a;i<b;i++)
#define fw(i,a,b) for(int i=a;i<=b;i++)
#define fd(i,b,a) for(int i=b;i>=a;i--)
#define nl '\n'
#define space ' '
#define int long long
#define N 207
#define yie9 1000000007

bool isp[N],in[N];
int n;
vector<int>v;

void init(){
    memset(isp,true,sizeof isp);
    for(int i=2;i<N;i++)for(int j=i;j*i<N;j++)isp[j*i]=false;
}
//上一个位置d填了x，现在要填第d+1个位置
void dfs(int x,int d){
```

```
    if(d==n){
        //填完n，特判是否为素数
        if(!isp[x+1])return;
        for(auto it:v)cout<<it<<space;
        cout<<nl;return;
    }
    fw(i,2,n){
        if(isp[x+i]&&!in[i]){
            v.push_back(i);
            in[i]=true;
            dfs(i,d+1);
            //回溯，清空当前状态
            v.pop_back();
            in[i]=false;
        }
    }
}
void solve(){
    v.push_back(1);
    int id=1;
    while(cin>>n){
        cout<<"Case "<<id++<<":"<<nl;
        dfs(1,1);
        cout<<nl;
    }
}
signed main(){
    ios::sync_with_stdio(0);cin.tie(0);

    init();
    //int t;cin>>t;while(t--)
    solve();
}
/*
*/
```

D - Open the Lock

这题寻找最小操作次数，然后我们注意到四位数，有效状态数量很少 (6561个) ，记作 n ,然后每个状态可以转化为若干个状态

我们可以把状态a转化到状态b视作在一个无向图中，a b两点中间有一条边

那么对于每次查询，我们可以在这个图上以a为中心进行BFS，直到找到b,此时经过的路径即为最短路径之一，复杂度$O(n)$（不建议使用dfs，这种情景下dfs会重复经过某些点多次）

当然也可以 01BFS 用 $O(n^2)$ 预处理两两数字之间的距离，然后 $O(1)$ 查询

代码：

```
#include<bits/stdc++.h>
using namespace std;
int num[5]={1,10,100,1000,10000};
int swa(int x,int i,int j){
    int a=x%num[i+1]/num[i],b=x%num[j+1]/num[j];
    x-=(num[i]-num[j])*a;
    x-=(num[j]-num[i])*b;
    return x;
}
int add(int x,int i){
    int a=x%num[i+1]/num[i];
    if(a==9){x-=num[i]*8;}
    else{x+=num[i];}
    return x;
```

```
}
int dec(int x,int i){
    int a=x%num[i+1]/num[i];
    if(a==1){x+=num[i]*8;}
    else{x-=num[i];}
    return x;
}
void solve(){
    int a,b;cin>>a>>b;
    vector<int>dis(10000,INT_MIN);
    list<int>bfs;bfs.push_back(a);dis[a]=0;
    while(!bfs.empty()){
        int x=bfs.front();bfs.pop_front();
        for(int i=0;i<4;i++){
            int newx=add(x,i);
            if(dis[newx]<0){dis[newx]=dis[x]+1;bfs.push_back(newx);}
        }
        for(int i=0;i<4;i++){
            int newx=dec(x,i);
            if(dis[newx]<0){dis[newx]=dis[x]+1;bfs.push_back(newx);}
        }
        for(int i=0;i<3;i++){
            int newx=swa(x,i,i+1);
            if(dis[newx]<0){dis[newx]=dis[x]+1;bfs.push_back(newx);}
        }
        if(dis[b]>=0){break;}
    }
    cout<<dis[b]<<"\n";
}
signed main(){
    ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);
    int t;cin>>t;
    while(t--){
        solve();
    }
}
```

E - 生日蛋糕

非常暴力的搜索和非常暴力的剪枝。

dfs 接受 4 个参数：

- 当前正在尝试累加的**层数** l
- 当前还需要填充的**体积** rv
- 当前已累计的**侧面积** rs
- 当前还没有放的**层数** z

以及我们使用数组 $r[], h[]$ 表示第 i 层圆柱的半径和高度。

暴力枚举

实际上剪枝已经开始了，每一层的半径和高度除了要比前一层小之外，还必须大于等于**剩余层数**，否则一定无法填满。

```
for(int i=r[l-1]-1; i>=z; i--) {
    for (int j=h[l-1]-1; j>=z; j--) {
        r[l]=i; h[l]=j;
        dfs(l+1, rv-i*i*j, rs+2*i*j, z-1);
    }
}
```

合法方案

当前剩余层数与待填充体积均为 0 时，即为合法。

```
if(z==0&&rv==0) {
    ans=min(rs+r[1]*r[1], ans); return;
}
```

失败

当任意剩余条件出现负数时一定非法。

```
if(z<0||rv<0) return;
```

剪枝

所谓剪枝，就是在**没有直接表明失败**而当前条件**注定无法成功**时，提前将不可能的分支扼杀在摇篮中，从而**大幅减少**递归深度。

当表面积不可能更优

当我们**乐观估计**当前分支的最优结果，即后续所有层的**半径与高度都为 1，即侧面积为 2** (这显然是不可能的)的情况下总面积也已经比我们其他分支得到的结果更大，就没有必要继续进行。

```
if(rs+2*z+r[1]*r[1]>ans) return;
```

当体积不可能达到 v

同理，当我们后续所有层都取得**当前层的半径与高度**(这显然也是不可能的)的情况下也无法填满剩余体积时，同样没有必要继续进行。

```
if(r[l-1]*r[l-1]*h[l-1]*z<rv) return;
```

最终代码

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
const int N=20;
int v, sl, r[25], h[25];
int ans=INT_MAX;
void dfs(int l, int rv, int rs, int z) {
    if(z==0&&rv==0) {
        ans=min(rs+r[1]*r[1], ans); return;
    }
    if(z<0||rv<0) return;
    if(rs+2*z+r[1]*r[1]>ans) return;
    if(r[l-1]*r[l-1]*h[l-1]*z<rv) return;

    for(int i=r[l-1]-1; i>=z; i--) {
        for (int j=h[l-1]-1; j>=z; j--) {
            r[l]=i; h[l]=j;
            dfs(l+1, rv-i*i*j, rs+2*i*j, z-1);
        }
    }
}
```

```
void solve() {
    cin>>v>>sl;
    r[0]=sqrt(v); h[0]=v;
    dfs(1, v, 0, sl);
    if (ans!=INT_MAX) cout<<ans<<endl;
    else cout<<0<<endl;
}
signed main() {
    ios::sync_with_stdio(0); cin.tie(0);
    solve();
    // int T; cin>>T; while(T--) solve();
    return 0;
}
```

F - 最大子树和

题意

给你一棵树，找到树上点权之和最大的一个连通分量，输出其权值之和。

思路

任选根节点进行 dfs，记录以当前节点为根节点的子树最大点权之和是多少。

问题在于，选定的根节点不一定在最后的结果中，所以我们可以在 dfs 的过程中，维护以每个节点为根节点的最大点权和。

code

```
#include <bits/stdc++.h>
using namespace std;

#define f(i,a,b) for(int i=a;i<b;i++)
#define fw(i,a,b) for(int i=a;i<=b;i++)
#define fd(i,b,a) for(int i=b;i>=a;i--)
#define nl '\n'
#define space ' '
#define int long long
#define N 200007
#define yie9 1000000007

vector<int> v[N];
int a[N],ans=-yie9;
//由于所有节点权值都可能为负，ans需要初始化为极小值。
void dfs(int x,int fa){
    for(auto it:v[x])
        if(it!=fa){
            dfs(it,x);
            if(a[it]>0)a[x]+=a[it];
            //实际上是个dp，不过也可以贪心地理解
        }
    ans=max(ans,a[x]);
}
void init(){}
void solve(){
    int n;cin>>n;
    fw(i,1,n)cin>>a[i];
    f(i,1,n){
        int a,b;cin>>a>>b;
        v[a].push_back(b);
        v[b].push_back(a);
```

```
    }
    dfs(1,0);
    cout<<ans<<nl;
}
signed main(){
    ios::sync_with_stdio(0);cin.tie(0);

    init();
    //int t;cin>>t;while(t--)
    solve();
}
/*
*/
```

G - 选课

首先，每个课程有且仅有一个先修课。接着，课程必须在所有先修课都完成的情况下才能学习
像什么呢？像dfs遍历树时候的特征：**每个节点有且仅有一个父节点，每个节点必须在父节点已经过的情况下才能到达**

考虑对题目中的课程建有向图，这个图的结构就必然是与一棵树高度相似的，不一样的是他有多个根

题目要求是获得最大学分，那么其实就是类似背包问题，这类问题以拿取为状态的话状态数量很大不好处理，通常用其他量（比如某负重下的最大价值，某价值下的最小负重）来作为主要状态
这题的数据特征是 课程总数量少，学分也小，所以从负重（课程数量）或者价值（学分）两个方向dp都是没问题的

那么以负重为例，我们设计一个dp表， dp[u][x] 表示dfs按序遍历到节点 u 并返回后，选x个课程的最大收益学分是多少

两个节点之间的状态转移就是

```
for(int v:g[u]){
    for(int i=dep;i<n;i++){
       dp[v][i]=dp[u][i-1]+val[v];
       dp[u][i]=max(dp[u][i],dp[v][i]);
    }
}
```

把他加进dfs的过程即可

由于有多个根，处理的时候可以分别从每个根出发，最后合并dp表，也可以创建一个虚根作为所有当前根的父节点，然后查询虚根dp表

时空复杂度 $O(n^2)$

代码：

```
#include<bits/stdc++.h>
using namespace std;
vector<vector<int>>nexts(400);
vector<int>roots;
vector<int>values(400);
void go(vector<int>&dp,int p){//实际上是dfs
    /*流程:
    得到一个所有位置都包含作为根节点的当前节点的dp表
    中序遍历子树根，下放处理的dp表
    最后比较dp表内的值，如果有比原表大的更新原表
    返回*/
    vector<int>newdp(dp.size(),INT_MIN);
    for(int i=dp.size()-1;i>=1;i--){
        newdp[i]=dp[i-1]+values[p];
    }
    for(int leaf:nexts[p]){
        go(newdp,leaf);
```

```
    }
    for(int i=0;i<dp.size();i++){
        dp[i]=max(dp[i],newdp[i]);
    }
}
int main(){
    ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);
    int n,m;cin>>n>>m;//n中选m个
    for(int i=1;i<=n;i++){
        int front;cin>>front>>values[i];
        if(front!=0){nexts[front].push_back(i);}
        else{roots.push_back(i);}
    }
    /*中序遍历然后在特定dp数组进行dp*/
    vector<int>dp(m+1,INT_MIN);
    dp[0]=0;
    for(int rt:roots){
        go(dp,rt);
    }
    cout<<dp[m];
}
```

H - Tree Jumps

本题有 bfs 和 dp 两种解法。

bfs

考虑**以某一个节点结尾**能产生的方案数为

- **其父节点所在深度的所有节点产生的总方案数-其父节点产生的方案数。**

所以本题需要我们按**层次**处理这棵树。可以联想到 bfs 即为**层序遍历**，对这棵树 bfs 的过程中分别维护每一个点的方案数与每一层的方案数即可。可以理解为当遍历到深度为 d 的节点时，深度为 d-1 的每个节点能产生的方案数一定已经统计完备了。

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
const int N=300005;
const int mod=998244353;
int n, fa[N], s[N], yuna[N], lv[N];
vector<int> g[N];
void bfs() {
    queue<int> q; q.push(1);
    while(!q.empty()) {
        int u=q.front(); q.pop();
        for(auto &v: g[u]) {
            s[v]=s[u]+1;
            if(s[v]==2) {
                lv[2]++; yuna[v]=1;
            }
            else {
                yuna[v]=(lv[s[v]-1]-yuna[u]+mod)%mod;
                lv[s[v]]=(lv[s[v]]+yuna[v])%mod;
            }
            q.push(v);
        }
    }
}
void solve() {
    cin>>n;
    for(int i=0; i<=n+1; i++) {
```

```
        s[i]=0; yuna[i]=0; lv[i]=0;
        g[i].clear();
    }
    fa[1]=0; s[1]=1;
    for(int i=2; i<=n; i++) {
        cin>>fa[i];
        g[fa[i]].push_back(i);
    }

    yuna[1]=1;
    bfs();

    int ans=0;
    for(int i=1; i<=n; i++) {
        ans=(ans+yuna[i])%mod;
    }

    cout<<ans<<'\n';
}
signed main() {
    ios::sync_with_stdio(0); cin.tie(0);
    // solve();
    int T; cin>>T; while(T--) solve();
    return 0;
}
```

dp

1. 注意到**同一深度的每个节点能产生的方案数相等**。
2. 同时又考虑到除了深度为 1 的节点以外，能对**以当前节点结尾能产生的方案数**产生贡献的点的数目为**其父节点所在深度所含节点数-1**。

所以我们可以跑一轮 dfs 求一下每个节点的深度，同时整理出每个深度所含的节点数目。直接在层次间 dp 就能求出答案，转移方程为

$$f[i] = f[i-1] \times (cd[i-1] - 1)$$

其中 $f[i]$ 表示以一个深度为 i 的节点为结尾能产生的方案数，$i > 1$

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
const int mod=998244353;
const int N=300005;
int n, fa[N], d[N], cd[N], f[N];
vector<int> g[N];
void getdeep(int u, int t) {
    d[u]=t; cd[t]++;
    for(auto &v: g[u]) getdeep(v, t+1);
}
void solve() {
    cin>>n;
    for(int i=0; i<=n; i++) {
        g[i].clear(); d[i]=cd[i]=f[i]=0;
    }
    for(int i=2; i<=n; i++) {
        cin>>fa[i]; g[fa[i]].push_back(i);
    }
    getdeep(1, 0);
    int ans=cd[0]+cd[1]; f[1]=1;
    for(int i=2; i<=n; i++) {
        f[i]=f[i-1]*(cd[i-1]-1)%mod;
        ans=(ans+cd[i]*f[i]%mod)%mod;
    }
    cout<<ans<<'\n';
```

```
}
signed main() {
    ios::sync_with_stdio(0); cin.tie(0);
    // solve();
    int T; cin>>T; while(T--) solve();
    return 0;
}
```

I - 树的直径

题意

求树的直径。

思路

没有边权，也就是边权均为 1，两次 dfs 即可。

第一次任选节点进行 dfs，距离他最远的节点一定在一条直径上，然后再从那个节点开始进行 dfs，求得的最远距离就是直径。

code

```
#include <bits/stdc++.h>
using namespace std;

#define f(i,a,b) for(int i=a;i<b;i++)
#define fw(i,a,b) for(int i=a;i<=b;i++)
#define fd(i,b,a) for(int i=b;i>=a;i--)
#define nl '\n'
#define space ' '
#define int long long
#define N 200007
#define yie9 1000000007

vector<int> v[N];
int e,d[N],mxd;
//e记录深度最大的节点
//进行了一个深度的变换，长度为n的路径上一定有n+1个点，即算深度，深度减1则是答案
void dfs(int x,int fa){
    d[x]=d[fa]+1;
    for(auto it:v[x])if(it!=fa)dfs(it,x);
    if(d[x]>mxd)mxd=d[x],e=x;
}
void init(){}
void solve(){
    int n;cin>>n;
    f(i,1,n){
        int a,b;cin>>a>>b;
        v[a].push_back(b);
        v[b].push_back(a);
    }
    dfs(1,0);
    dfs(e,0);
    cout<<mxd-1<<nl;
    //由于根节点深度为1，输出需要减去1
}
signed main(){
    ios::sync_with_stdio(0);cin.tie(0);
```

```
    init();
    //int t;cin>>t;while(t--)
    solve();
}
/*
*/
```

J - 最近公共祖先

最经典的做法就是倍增，存储每个点往根方向走 1,2,4,8,16,32····2^20 次方距离所到达的点

处理节点 u 走 2^n 步会走到哪里的时候，先找到走 2^{n-1} 步所到达的节点 q ，从 q 节点走 2^{n-1} 步所到达的点，就是 u 走 2^n 步所到达的节点

查询lca的时候,先让深度浅一直走到深度相同，然后两点循环往根的方向走，每次至少走一步，直到两点相遇，这个相遇点就是要求的LCA

时间复杂度上，预处理 ***O(nlogn)***，每次查询 ***O(logn)***，空间复杂度 ***O(nlogn)*** 。**常数比较大**

然后有另外一种类似的做法。记根的深度为0，每个深度为的节点依次维护自己往根方向走 ***0,1,3,7,15*** (其实就是从小到大排的 2^n***-1***)距离所到达的点，直到这个距离大于自身深度，距离重置为 0 ，然后为了方便查询，把 0 记为 1

听上去很麻烦但是代码其实很简单

lb维护的是走 2^n-1 距离到达的点，fa存储父节点

参考代码：

```
void dfs(int u){
    int p=fa[u],q=lb[p],r=lb[q];
    d[u]=d[p]+1;
    lb[u]=(d[p]-d[q])!=(d[q]-d[r])?p:r;
    for(int v:g[u]){
        if(v!=p){fa[v]=u;dfs(v);}
    }
}
int lca(int u,int v){
    if(d[u]<d[v])swap(u,v);
    while(d[u]>d[v]){
        if(d[lb[u]]<d[v]){
            u=fa[u];
        }else{
            u=lb[u];
        }
    }
    while(u!=v){
        if(lb[u]==lb[v]){
            u=fa[u];v=fa[v];
        }else{
            u=lb[u];v=lb[v];
        }
    }
    return u;
}
```

预处理时间复杂度 ***O(n)*** ,空间复杂度 ***O(n)*** ,查询的最劣时间复杂度 ***O(logn*logn)*** ，均摊时间复杂度 ***O(logn)***

这个方法最大的优点是 **常数非常小** ，通常运行起来比倍增更快（也可以用于替换某些树上倍增的逻辑，但不一定优而且容易写挂，不推荐这么干）

上述的方法都属于在线算法，支持一定程度的动态修改，下面简单介绍几种离线的做法，详细的请查阅资料

比较推荐的是树链剖分，LCA为两个游标跳转到同一条重链上时深度较小的那个游标所指向的点
O(n) 预处理，***O(logn)*** 查询，常数小，实现也相对容易

可以用tarjan算法和并查集结合去做，这里不多讲，有兴趣自己查找相关资料。
时间复杂度有 $O(m\alpha(m+n,n)+n)$ 的，也有严格 ***O(n+m)*** 的（实现比较复杂），α表示反阿克曼函数，通常是比logn更小的值,可以近似为常数

还有一种利用欧拉序的性质，可以把树上的LCA问题***转化为RMQ问题***，然后利用RMQ问题的解法（比如st表）去做。

K - STA-Station

这是一个最值问题。我们把问题拆分一下。

首先考虑一棵**有根树**的所有节点深度之和的求法。不难想到只要从根节点开始进行 dfs，递归的找出每一个的深度 $d[i]$ 再进行求和即可，复杂度为 $O(n)$.

而本题要求我们找出使这个值最大的点。最朴素的思想是枚举每一个点 i，依次指定 i 为根进行上述操作，然而这样的复杂度来到了 $O(n^2)$.

树上问题很多都是将无根转化为有根。因为有根树的父子关系确定，只需要进行一轮 dfs。希望将多根问题转化为单根问题，我们的思路本质上是做这样一件事：

- 对于一对父子节点 u, v，已知以 u 为根的信息 $f[u]$，在不显式换根的前提下，尝试使用 $f[u]$ 推导出 $f[v]$。
- 如果可以转移，**将信息不断地从父节点递推到子节点**，进而推导出全图所有点的信息。这个过程与朴素 dfs 一致。

在本题中 $f[i]$ 即为**以 i 为根节点时所有节点的深度之和**。

于是问题转化为

- **如何推导出与根节点 u 直接相连的节点 v 的** $f[v]$.

这是一个相对简单的问题。在当 v 变为根节点时，与其相连的所有节点(包括 v 自身)的深度都会减一，而除了这些节点以外的所有节点深度(包括 u 和与 u 相连的其他节点)都会加一，那么这个结果即为

$$f[v] = f[u] + (n - cnt[v]) - cnt[v]$$

其中 $cnt[i]$ 表示在不换根时 i 的子树大小。所以在第一次 dfs 求每个点的深度时，同时求一下每个点的子树大小即可。与深度不同，这个过程是自下而上的。

这种思想也被称为**换根法**。

```
#include<bits/stdc++.h>
using namespace std;
const int N=1000005;
int n, cnt[N], d[N], ans;
long long f[N], sum, mx;
vector<int> g[N];
void dfs(int u, int fa) {
    d[u]=d[fa]+1; cnt[u]=1;
    for(auto &v: g[u]) {
        if(v!=fa) {
            dfs(v, u);
            cnt[u]+=cnt[v];
        }
    }
}
void dp(int u, int fa) {
    for(auto &v: g[u]) {
        if(v!=fa) {
            f[v]=f[u]+n-2*cnt[v];
            if(f[v]>mx) mx=f[v], ans=v;
```

```
            dp(v, u);
        }
    }
}
void solve() {
    cin>>n;
    for(int i=1; i<n; i++) {
        int u, v; cin>>u>>v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    dfs(1, 0);
    for(int i=1; i<=n; i++) {
        f[1]+=d[i];
    }
    mx=f[1], ans=1;
    dp(1, 0);
    cout<<ans;
}
signed main() {
    ios::sync_with_stdio(0); cin.tie(0);
    solve();
    // int T; cin>>T; while(T--) solve();
    return 0;
}
```

L - Trapmigiano Reggiano

题意

给定一棵树，老鼠从 st 出发。

要求一个排列p，每步老鼠会向 p_i 方向沿最短路走一条边。

要求 n 步结束时老鼠一定停在 en，问能否构造这样的排列。

思路

只要老鼠一直围绕着目标节点走就可以。

也就是说，以 en 为根，老鼠的下一个目标在 en 的其他子树中。

由于树上的路径是唯一的，所以可以保证老鼠一定会向 en 移动。

具体而言，以目标节点为根节点进行 bfs，分配奶酪出现的位置。

由于是倒序 bfs 的，所以需要将得到的序列逆序输出，这样按时间顺序会将老鼠引到 en。

code

```
#include <bits/stdc++.h>
using namespace std;

#define f(i,a,b) for(int i=a;i<b;i++)
#define fw(i,a,b) for(int i=a;i<=b;i++)
#define fd(i,b,a) for(int i=b;i>=a;i--)
#define nl '\n'
#define space ' '
#define int long long
```

```
#define N 200007
#define yie9 1000000007

struct node{
    int x,fa;
};
void init(){}
void solve(){
    int n,st,ed;cin>>n>>st>>ed;
    vector<int> v[n+1],ans;
    f(i,0,n-1){
        int a,b;cin>>a>>b;
        v[a].push_back(b);
        v[b].push_back(a);
    }
    queue<node>q;
    q.push({ed,0});
    while(q.size()){
        node tp=q.front();q.pop();
        ans.push_back(tp.x);
        for(auto it:v[tp.x])if(it!=tp.fa){
            q.push({it,tp.x});
        }
    }
    fd(i,n-1,0)cout<<ans[i]<<space;
    cout<<nl;
}
signed main(){
    ios::sync_with_stdio(0);cin.tie(0);

    init();
    int t;cin>>t;while(t--)
    solve();
}
/*
*/
```

M - 奥日

简单来说，题目要求找树上最长的路径，使得重复经过点的次数不超过k次，且经过的点数量最大

首先考虑对一条链状路，如果我们想要在中间侧增加分支点，每增加一个点就会使得重复经过的次数+1，前后增加点则无代价开销

那么我们要做的就是找到一条最长的链状路径，然后在它的基础上添加k个分支点，而这个最长的路径就是树的直径。这是一道找树的直径的模板题。

直径有个性质，就是从树上任意一点出发dfs，能抵达的最远的点一定是树的某条直径的一个端点，利用这个性质跑两轮dfs就能得到直径，复杂度 ***O(n)***

代码：

```
#include<bits/stdc++.h>
using namespace std;
vector<vector<int>>g;
vector<int>de;
int n,k;
int maxDepth=0,deep=-1;
void depthdfs(int u,int d){
    if(d>maxDepth){maxDepth=d;deep=u;}
    de[u]=d;d++;
    for(int v:g[u]){
        if(!de[v]){depthdfs(v,d);}
    }
```

```
}
vector<int>pa;
int md=0,theb=-1;
void dfs2(int u,int d){
    if(d>md){md=d;theb=u;}
    for(int v:g[u]){
        if(v!=pa[u]){
            pa[v]=u;dfs2(v,d+1);
        }
    }
}
vector<bool>vis;
vector<int>ans;
void wayBuild(int u){
    ans.push_back(u);vis[u]=true;
    if(u==deep){return ;}
    for(int v:g[u]){
        if((!vis[v])&&k>0&&(v!=pa[u])){k--;wayBuild(v);}
    }
    if(!vis[pa[u]]){wayBuild(pa[u]);return;}
    else{ans.push_back(pa[u]);}
}
int main(){
    ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);
    int t;cin>>t;
    while(t--){cin>>n>>k;
        g.resize(n+1);de.resize(n+1);pa.resize(n+1);vis.resize(n+1);
        for(int i=1;i<n;i++){
            int u,v;cin>>u>>v;
            g[u].push_back(v);
            g[v].push_back(u);
        }
        depthdfs(1,1);
        dfs2(deep,1);
        wayBuild(theb);
        cout<<ans.size()<<"\n";
        for(int a:ans){cout<<a<<" ";}cout<<"\n";
        cout.flush();
        g.clear();
        de.clear();
        pa.clear();
        vis.clear();
        ans.clear();maxDepth=0;deep=-1;md=0;theb=-1;
    }
}
```

N - 松鼠的新家

本题分为两部分，分别是寻找若干对点之间的简单路径，以及对路径上每个点的值进行标记。这些标记是可重叠的，换句话说，当我们重复经过某个点时标记会层层叠加，记录下该点被经过的总次数。

寻找简单路径

树有一个很强的性质，即

- 树上任意两点之间有且仅有一条简单路径。

而我们又知道在**有根树上两点**间有一个东西叫**最近公共祖先**，品一品就会发现分别**连接它们与它们的最近公共祖先所形成的路径一定是一条简单路径**。如果你还没有掌握 LCA 算法，请点击这里。

所以我们得到了一个结论，寻找在**无根树**上的**若干对点间的简单路径**可以在任意一棵**有根树**上进行。

标记路径上的点

如何标记这条边呢？对于这种类似于**区间修改**的操作，我们容易联想到差分。

差分

序列上差分

序列上差分简单来说就是记录一个差分数组。差分数组可以将**区间加减**降维成**单点修改**。通过记录相邻元素的差值，我们将每一次操作累加到差分数组中，最后执行一次前缀和还原。如果你还没有掌握序列上差分，请点击这里。

树上差分

和序列上差分类似，我们可以在**有根树**的节点上打上标记，在 dfs 的过程中依次更新这些值。

举个例子，但我们想要标记 6 到 8 之间的路径时，我们可以给 6 和 8 打上+1，给它们的 **LCA** 3 和 **LCA 的父节点** 2 打上 **-1**。当我们自下而上更新每个节点的值时，这种标记方法可以达到与序列上差分类似的效果。

```
          1
          |
          2   (-1)
          |
          3   (-1)
         / \
        4   5
       / \   \
  (+1)6  7    8(+1)
```

本题所使用的是树上**点差分**，若要对经过的边进行标记，将使用树上**边差分**，与点差分类似，请同学们自行学习。

回到本题

题目要求我们按照一定顺序访问一些节点，实际上我们的路径由该数组内每对相邻元素组成。

值得注意的点是，两条路径的**公共点**只应该被标记一次，再加上题目所提到的最后经过的点不需要+1，不难发现其意在引导我们在**每条路径**的**起点**和**终点的父节点**打上+1，而不是起点和终点。

```cpp
#include<bits/stdc++.h>
using namespace std;
#define int long long
const int N=300005;
int n, a[N], d[N], f[N][20], ta[N], cnt[N];
vector<int> g[N];
void ycl(int u, int fa, int dd) {
    d[u]=dd;
    f[u][0]=fa;
    for(int i=1; i<=19; i++) {
        f[u][i]=f[f[u][i-1]][i-1];
    }
    for(auto &v: g[u]) {
        if(v!=fa) ycl(v, u, dd+1);
    }
}
int lca(int u, int v) {
    if(d[u]<d[v]) swap(u, v);
    for(int i=19; i>=0; i--) {
        if(d[f[u][i]]>=d[v]) u=f[u][i];
    }
    if(u==v) return u;
    for(int i=19; i>=0; i--) {
        if(f[u][i]!=f[v][i]) {
            u=f[u][i], v=f[v][i];
```

```
        }
    }
    return f[u][0];
}
void draw(int u, int fa) {
    cnt[u]=ta[u];
    for(auto &v: g[u]) {
        if(v!=fa) {
            draw(v, u);
            cnt[u]+=cnt[v];
        }
    }
}
void solve() {
    cin>>n;
    for(int i=1; i<=n; i++) {
        cin>>a[i];
    }
    for(int i=1; i<n; i++) {
        int u, v; cin>>u>>v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    ycl(1, 0, 1);
    for(int i=1; i<n; i++) {
        int u=a[i], v=a[i+1];
        int yu=lca(u, v);
        ta[f[yu][0]]--;
        ta[yu]--;
        ta[u]++; ta[f[v][0]]++;
    }
    draw(1, 0);
    for(int i=1; i<=n; i++) {
        cout<<cnt[i]<<'\n';
    }
}
signed main() {
    ios::sync_with_stdio(0); cin.tie(0);
    solve();
    // int T; cin>>T; while(T--) solve();
    return 0;
}
```

O - 仓鼠找 sugar

题意

求树上两条路径是否有交点。

思路

若两条路径有交点，则一条路径的 lca 一定在另一条路径上。

问题转化为了如何判断树上一点在某条路径上。

由于树上路径唯一，只需要判断该点到两端点的距离之和是否等于两点之间的距离即可。

则只需求出树上两端点的距离。

从树根出发，他们的深度一定是根节点到 lca 的距离+ lca 到端点的距离。

则两点之间的距离为他们的深度之和减去两倍 lca 的深度。

code

```
#include <bits/stdc++.h>
using namespace std;

#define f(i,a,b) for(int i=a;i<b;i++)
#define fw(i,a,b) for(int i=a;i<=b;i++)
#define fd(i,b,a) for(int i=b;i>=a;i--)
#define nl '\n'
#define space ' '
#define int long long
#define N 200007
#define yie9 1000000007

vector<int> vn[N];
int fa[N][20],d[N];
void dfs(int x,int ft){
    d[x]=d[ft]+1,fa[x][0]=ft;
    for(auto it:vn[x])if(it!=ft)dfs(it,x);
}
int lca(int u,int v){
    if(u==v)return u;
    if(d[u]<d[v])swap(u,v);
    fd(i,18,0)if(d[fa[u][i]]>=d[v])u=fa[u][i];
    if(u==v)return u;
    fd(i,18,0)if(fa[u][i]!=fa[v][i])u=fa[u][i],v=fa[v][i];
    return fa[u][0];
}
int dis(int x,int y){
    return d[x]+d[y]-2*d[lca(x,y)];
}
void init(){}
void solve(){
    int n,q;cin>>n>>q;
    f(i,1,n){
        int a,b;cin>>a>>b;
        vn[a].push_back(b);
        vn[b].push_back(a);
    }
    dfs(1,0);
    fa[1][0]=1;
    fw(i,1,18)fw(j,1,n)fa[j][i]=fa[fa[j][i-1]][i-1];
    while(q--){
        int a,b,x,y;cin>>a>>b>>x>>y;
        int l1=lca(a,b),l2=lca(x,y);
        if(dis(l1,x)+dis(l1,y)==dis(x,y)||dis(l2,a)+dis(l2,b)==dis(a,b))cout<<"Y"<<nl;
        else cout<<"N"<<nl;
    }
}
signed main(){
    ios::sync_with_stdio(0);cin.tie(0);

    init();
    //int t;cin>>t;while(t--)
    solve();
}
/*
*/
```

P - Choosing the Capital of Treeland

题目要求找到一个点使得有向图能在改最少的边的方向的情况下，从所有点可达该点

注意到题目所给的n-1条边和n个点构成一个树形结构

已知性质：树上任意一点或者任意一条边都能把树划分为两个部分

于是，如果已知一个点作为汇聚中心时候的代价，那么它相邻的点作为汇聚中心的代价就是它的代价 **±1**，具体根据边的原始方向决定。一个非常简单的换根dp思路

我们考虑创建一个无向图，然后保存下来具体方向，先随便找一个点作为汇聚中心，然后该点作为汇聚中心，然后从这个点出发dfs，就能n次 ***O(1)*** 的预处理出来所有点作为汇聚中心时的代价

那么这题就做完了，时空复杂度 ***O(n)***

代码：

```
#include<bits/stdc++.h>
using namespace std;
vector<vector<int>>g;
set<pair<int,int>>dir;
vector<int>dep;
int toroot=0;
vector<vector<int>>ins;
void dfs(int u,int d){
    dep[u]=d;
    for(int v:g[u]){
        if(!dep[v]){dfs(v,d+1);}
        else if(dir.count(make_pair(u,v))){
            toroot++;
        }
    }
}
vector<bool>vis;
int minin=INT_MAX;
void dfs2(int u,int in){
    vis[u]=true;
    ins[in].push_back(u);minin=min(in,minin);
    for(int v:g[u]){
        if(!vis[v]){
            if(dir.count(make_pair(u,v))){dfs2(v,in+1);}
            else{dfs2(v,in-1);}
        }
    }
}
int main(){
    ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);
    int n;cin>>n;
    g.resize(n+1);dep.resize(n+1);vis.resize(n+1);ins.resize(n+1);
    for(int i=1;i<n;i++){
        int u,v;cin>>u>>v;
        dir.insert(make_pair(u,v));
        g[u].push_back(v);g[v].push_back(u);
    }
    dfs(1,1);
    dfs2(1,toroot);
    cout<<minin<<endl;
    sort(ins[minin].begin(),ins[minin].end());
    for(int u:ins[minin]){
        cout<<u<<" ";
    }cout<<endl;
}
```

Q - Serval and Rooted Tree

这是一道思维难度显著大于实现难度的题。题目的意思大致是：

- 我们拥有一棵**有根树**，每一个**非叶子节点**都拥有一种操作 MAX 或 MIN，我们可以将 1～k 这 k 个值安排在 k 个**叶子节**

点上，求在**根节点**处取能得的最大值。

从结果来看，我们假设最终能取到的最大值为 x，意思是我们无法通过调整叶子节点值的安排方式使得根节点得到的值大于 x. 这句话中隐含的条件是：

- 我们最少需要消耗 k-x+1 个大于等于 x 的值(包括其自身)来得到第 k-x+1 大的值 x ，且任何不存在代价比 k-x+1 更小的方案。

很好证明，如果存在一个代价 k-x+1-d 的方案，那么最终答案可以取到 1~k 中第 k-x+1-d 大的值 x+d，这是一个更大的值，矛盾。

所以再换句话说，这道题在找的是**最少代价**，即

- 要使得根节点取得 x，最少花费多少个**大于等于 x** 的值。

在**以 x 为基准**的视角下，在处理每一个 **MIN** 和 **MAX** 操作时，可以对 1～k 每个数进行归类：

1. 所有**比 x 小的值**都可以视作 **0**
2. 所有**大于等于 x 的值**都可以视作 **x**

那么问题最终转化为我们**最少需要在多少个叶子节点上放上 x** (其余叶子节点均为0)，可以使得根节点取到 x. 最终问题就变成了一个 dp.

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
const int N=300005;
int n, a[N], f[N], ans;
vector<int> g[N];
void dfs(int u, int fa) {
    if(g[u].size()==0) {
        ++ans;
        f[u]=1; return;
    }
    int sum=0, mn=LLONG_MAX;
    for(auto &v: g[u]) {
        if(v!=fa) {
            dfs(v, u);
            sum+=f[v];
            mn=min(mn, f[v]);
        }
    }
    if(a[u]) f[u]=mn;
    else f[u]=sum;
}
void solve() {
    cin>>n;
    for(int i=1; i<=n; i++) {
        cin>>a[i];
    }
    for(int i=2; i<=n; i++) {
        int t; cin>>t;
        g[t].push_back(i);
    }
    dfs(1, 0);
    cout<<ans-f[1]+1;
}
signed main() {
    ios::sync_with_stdio(0); cin.tie(0);
    solve();
    // int T; cin>>T; while(T--) solve();
    return 0;
}
```

R - Kay and Snowflake

题意

给定一棵树，询问某节点的重心。

思路

数据范围 $3e5$，考虑预处理出所有的答案，递推出所有子树的重心。

重心一定在以重儿子为根的子树的重心的连线上，也就是说，当前节点的重心一定是从最大的子树转移上来的，特判当前节点是不是重心，如果不是，从最大子树的重心不断向上跳，判断子树大小即可。

code

```
#include <bits/stdc++.h>
using namespace std;

#define f(i,a,b) for(int i=a;i<b;i++)
#define fw(i,a,b) for(int i=a;i<=b;i++)
#define fd(i,b,a) for(int i=b;i>=a;i--)
#define nl '\n'
#define space ' '
#define int long long
#define N 300007
#define yie9 1000000007

vector<int> v[N];
int fa[N],ct[N],sz[N];
void init(){}
void dfs(int x,int ft){
    sz[x]=1;int mxs=0;
    for(auto it:v[x])if(it!=ft){
        dfs(it,x);sz[x]+=sz[it];
        if(sz[it]>=sz[mxs])mxs=it;
    }
    //如果最大的子树的大小也小于sz/2，说明我自己就是重心。
    if(sz[mxs]*2<sz[x])ct[x]=x;
    else{
        int node=ct[mxs];
        while(sz[node]*2<sz[x])node=fa[node];
        //如果我的子树小于sz/2，则说明不是重心，继续向上跳跃。
        ct[x]=node;
    }
}
void solve(){
    int n,q;cin>>n>>q;
    fw(i,2,n){
        int ft;cin>>ft;
        v[ft].push_back(i);
        fa[i]=ft;
    }
    dfs(1,0);
    while(q--){
        int x;cin>>x;
        cout<<ct[x]<<nl;
    }
}
signed main(){
    ios::sync_with_stdio(0);cin.tie(0);
```

```
    init();
    //int t;cin>>t;while(t--)
    solve();
}
/*
*/
```

S - Ranges on Tree

这道题目的正确构造很简单，给所有叶子节点赋予 [x,x] 样子的区间，然后在树上合并即可

这里介绍一下为什么这么做。

首先介绍一下欧拉序

简单来说，你dfs的时候每个的进出都有时间先后顺序，那么把这个进出事件按照时间先后顺序记录下来，就成了欧拉序

比如对于一棵根为1，有边 (1 2),(2 3),(1 4)的树，他的欧拉序就是 ***1 2 3 3 2 4 4 1***
换种方式写就是 ***1(in) 2(in) 3(in) 3(out) 2(out) 4(in) 4(out) 1(in)*** ，in表示遍历第一次来到这个点，out表示离开这个点

不难发现，对于根节点 1 ，所有包含在它子树的点，欧拉序中都在两次 1 之间出现，这个是由dfs的性质决定的，可以推广到任意情况的性质

那么回到题目的区间，我们要让所有子树中点对应的区间，都被子树的根节点包含，是不是很像欧拉序的包含?

于是我们就有了欧拉序方向的思路。对于题目样例2，欧拉序就是 ***1 2 2 4 3 3 5 5 4 1***

问题转化到欧拉序上之后，我们就是只要保证对于所有在欧拉序中形如 ***... a ... b ... b ... a ...*** 的情况，区间 a 真包含区间 b

那么如果内层已经定下来，外层的最优情况就是区间并集，很容易得到也很容易想到
那么对于特殊情况 ***a...bb...a***，b已经是最内层了，如何确认一个最优的b呢?

其实答案就藏在欧拉序的连续性。我们只要根据这个连续性，来让 bb 对应的区间在欧拉序上连续分布即可，所有 bb 类型的依次分配 ***(1,1),(2,2)...*** 这样子就可以保证区间尽可能小

那么bb是什么呢? 其实就是叶子的欧拉序对应序列

所以做法就是dfs遍历，对于每个叶子按照遍历到的顺序依次赋予(1,1),(2,2),(3,3)····的区间即可

代码:

```
#include<bits/stdc++.h>
using namespace std;
int l[214514],r[214514];
vector<bool>vis(214514);
vector<int>g[214514];
int pointed=0;
void dfs(int u){
    l[u]=INT_MAX,r[u]=INT_MIN;
    vis[u]=true;
    for(int v:g[u]){
        if(!vis[v]){
            dfs(v);
            l[u]=min(l[u],l[v]);
            r[u]=max(r[u],r[v]);
        }
    }
    if(l[u]>r[u]){//说明是叶子
        pointed++;
        l[u]=pointed;
        r[u]=pointed;
    }
```

```
}
signed main(){
    ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);
    int n;cin>>n;
    for(int i=1;i<n;i++){
        int u,v;cin>>u>>v;
        g[u].push_back(v);g[v].push_back(u);
    }
    dfs(1);
    for(int i=1;i<=n;i++){
        cout<<l[i]<<" "<<r[i]<<" \n";
    }
}
```

T - LIS on Tree

本题为树上 LIS 问题，如果你还没有掌握序列上 $O(nlogn)$ 的 LIS 算法(贪心+二分)，请点击这里。

这是一道非常精巧的题，巧妙利用了 dfs **回溯**的性质将 LIS 直接套在了树上。

序列上 LIS

序列上 LIS 可以很容易地维护。从左到右扫一遍，整理出最终的 LIS 数组，其长度即为整个序列的 LIS 长度。不仅如此，实际上在枚举到任意位时，我们都可以知道**当前前缀**可以形成的 LIS 长度。

换句话说，假如说现在有一道题这样出

- 求出序列左端点到**每一个 i** 的区间内的 LIS 长度

那我们只要在枚举完每个 i 时输出当前 LIS 数组长度即可。本题和这一题唯一的区别就是这个过程发生在树上。

树上 LIS

其实**有根树**完全可以看成**若干条具有公共前缀的序列**。而对有根树的 dfs 过程即为对所有序列遍历的过程。

我们结合具体的过程来看。当我们开始 dfs 时，我们会沿着树的最左边的一条链走到最左边的叶子节点，目前来说这个过程中的每一个点都和序列上 LIS 没有任何区别。当我们递归到叶子节点开始向上回溯时，实际上我们可以同时**回溯我们对 LIS 数组的操作**。

具体来说：

- 假如我们向下通过当前节点时**改变**了 LIS 数组中的某一位，在向上回溯时我们就把这一位**还原**。
- 假如我们向下通过当前节点时在 LIS 数组的末尾**添加**了一个数，在向上回溯时我们就把这个数**删除**。

所以说，当我们通过回溯再一次到达某个点时，我们都可以**重新得到第一次经过这个点时的 LIS 数组的状态**，这样我们就成功地把树上 LIS 转化为了若干条链上的 LIS。

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
const int N=300005;
int n, a[N], f[N], idx, t, yuna[N];
vector<int> g[N];
bool check(int x) {
    if(f[x-1]<t) return 1;
    else return 0;
}
int ff(int l, int r) {
    if(l+1>=r) return l;
```

```
    int mid=l+r>>1;
    if(check(mid)) return ff(mid, r);
    else return ff(l, mid);
}
void dfs(int u, int fa) {
    t=a[u];
    if(f[idx]<t) {
        f[++idx]=t; yuna[u]=idx;
        for(auto &v: g[u]) {
            if(v!=fa) dfs(v, u);
        }
        --idx;
    }
    else {
        yuna[u]=idx;
        int pos=ff(1, idx+1);
        int stay=f[pos];
        f[pos]=t;
        for(auto &v: g[u]) {
            if(v!=fa) dfs(v, u);
        }
        f[pos]=stay;
    }
}
void solve() {
    cin>>n;
    for(int i=1; i<=n; i++) {
        cin>>a[i];
    }
    for(int i=1; i<=n; i++) {
        int u, v; cin>>u>>v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    f[0]=-1; idx=0;
    dfs(1, 0);
    for(int i=1; i<=n; i++) {
        cout<<yuna[i]<<'\n';
    }
}
signed main() {
    ios::sync_with_stdio(0); cin.tie(0);
    solve();
    // int T; cin>>T; while(T--) solve();
    return 0;
}
```

U - Dynamic Diameter

题意

给定一棵 n 个点的树，以及一个孤立点 $n+1$。
对每个点 i，把 $n+1$ 连到 i，问当前情况下新树的直径是多少。

思路

只有新节点接在直径的端点上时，才会对整棵树的直径长度造成影响。

所以只需要判断节点是否接到了直径的端点上。

边权为 1，考虑求直径的方法，两次 dfs，第一次求出距离任意节点最远的点的距离，该节点一定在直径上，再以此节点为根节点进行 dfs，求出另一个端点。

只需要记录第一次求出的距离为 max 的所有节点，和第二次 dfs 求出的所有距离为直径的节点即可。

code

```
#include <bits/stdc++.h>
using namespace std;

#define f(i,a,b) for(int i=a;i<b;i++)
#define fw(i,a,b) for(int i=a;i<=b;i++)
#define fd(i,b,a) for(int i=b;i>=a;i--)
#define nl '\n'
#define space ' '
#define int long long
#define N 200007
#define yie9 1000000007

vector<int> v[N];
int d[N],mxd,id;
bool ad[N];
void dfs(int x,int fa){
    d[x]=d[fa]+1;mxd=max(mxd,d[x]);
    for(auto it:v[x])if(it!=fa)dfs(it,x);
}
void init(){}
void solve(){
    int n;cin>>n;
    f(i,0,n-1){
        int a,b;cin>>a>>b;
        v[a].push_back(b);
        v[b].push_back(a);
    }
    dfs(1,0);
    fw(i,1,n)if(d[i]==mxd)ad[i]=true,id=i;
    dfs(id,0);
    fw(i,1,n)if(d[i]==mxd)ad[i]=true;
    fw(i,1,n)if(ad[i])cout<<mxd<<nl;
    else cout<<mxd-1<<nl;
}
signed main(){
    ios::sync_with_stdio(0);cin.tie(0);

    init();
    //int t;cin>>t;while(t--)
    solve();
}
/*
*/
```

V - Disrupting Communications

注：这题虽然题解写的比较短，但是只是因为概念已经在前面介绍过，没有重复介绍，实际实现比较复杂

首先题目要我们求的东西奇形怪状的，很难搞，暴力处理$O(n^2)$绝对会爆，我们考虑简化问题
考虑好搞的情况，最好搞得就是一对查询u，v中，u是我们的树的树根

那么答案就是u到v路径上每个点的子树中包含这个根的子图的数量的总和

这个可以用树形dp来$O(n)$预处理出每个点对应子树中的数量，记作dp_root,对于根i,子节点有$j1, j2, \cdots jk$,那么
$dp_root = \prod_{i=1}^{n} dp_root[x] + 1$

然后求和的过程，就用树上倍增，和倍增法lca的原理完全相同，不过转移变成
$dp_r[u][i] = dp_r[u][i-1] + dp_r[jumpPos][i-1]$;

单次查询是logn

那么考虑能不能全部转化为这种子树的问题呢？可以。
我们只要把u,v的lca找出来，lca就能把这个链状路径划分到两个子树里面
然后就变成了

查询结果 $=$ 链路$u->lca$的贡献 $+$ 链路$v->lca$的贡献 $+$ 所有包含lca的子图的数量

所有包含lca的子图的贡献很好处理，做一次以原始根节点为起点的换根dp即可

然后特别需要注意的是，换根dp的时候求积如果用到了逆元，需要特别处理 $(dp_root[u]+1)\%mod=0$ 的情况

代码：

```
#include<bits/stdc++.h>
#define int long long
using namespace std;
/*这题是不是要先划分要求的值？*/
/*
考虑每对点，子图可以分为包含LCA的和不包含LCA的，这样子就不需要考虑父节点方向的事情了
包含LCA的部分很好求，进行一轮预处理就行了
另外一部分就变成子树中包含根到某一节点
root_dp[i]设置为以某点为根它的子树的子图数量
core_dp[i]设置为以某点为中心它的子树的子图数量

*/
int mod=998244353;
int fastmi(int x,int y){
    int sum=1;
    while(y){
        if(y&1){sum*=x;sum%=mod;}
        x*=x;x%=mod;
        y>>=1;
    }
    return sum;
}
int n,q;
vector<int>g[114514],dis[114514],pa(114514);
int r_dp[114514][20],jump[114514][20];
int root_dp[114514],z[114514],core_dp[114514],dep[114514];
void dfs(int u){
    dis[dep[u]].push_back(u);
    root_dp[u]=1;
    for(int v:g[u]){
        dep[v]=dep[u]+1;
        dfs(v);
        /*后续需要对乘积乘逆元来还原乘积，因此对无法求逆元的0得特殊处理*/
        int valv=z[v]?0:root_dp[v];
        if(((valv+1)%mod)==0){
            z[u]++;
        }else{
            root_dp[u]*=valv+1;
            root_dp[u]%=mod;
        }
    }
}
void getCore(int u,int up){
    core_dp[u]=root_dp[u]*(up+1)%mod;
    for(int v:g[u]){
        int newup=core_dp[u];
        int valv=z[v]?0:root_dp[v];
        if(z[u]==0){
            newup*=fastmi((valv+1)%mod,mod-2);
            newup%=mod;
        }else{
            if(z[u]==1&&((valv+1)%mod==0)){}
            else{newup=0;}
```

```
        }
        getCore(v,newup);
    }
}
void dealRdp(){
    for(int d=0;d<n;d++){
        for(int u:dis[d]){
            r_dp[u][0]=z[u]?0:root_dp[u];
            jump[u][0]=pa[u];
            for(int i=1;i<20;i++){
                if(d<(1<<i)){break;}
                else{
                    int midPos=jump[u][i-1];
                    jump[u][i]=jump[midPos][i-1];
                    r_dp[u][i]=r_dp[u][i-1]+r_dp[midPos][i-1];
                    r_dp[u][i]%=mod;
                }
            }
        }
    }
}
int lca(int u,int v){
    if(dep[u]>dep[v]){swap(u,v);}//保证u比v更接近根
    for(int i=19;i>=0;i--){//处理到dep[u]==dep[v]
        if(dep[v]-(1<<i)>=dep[u]){
            v=jump[v][i];
        }
    }
    while(u!=v){
        for(int i=1;i<20;i++){
            if(dep[u]<(1<<i)||jump[u][i]==jump[v][i]){
                u=jump[u][i-1];v=jump[v][i-1];
                break;
            }
        }
    }
    return u;
}
void solve(){
    cin>>n>>q;
    for(int i=0;i<=n;i++){
        g[i].clear();dis[i].clear();pa[i]=0;
        root_dp[i]=0;z[i]=0;core_dp[i]=0;dep[i]=0;
        for(int j=0;j<20;j++){
            jump[i][j]=0;r_dp[i][j]=0;
        }
    }
    for(int i=2;i<=n;i++){
        cin>>pa[i];
        g[pa[i]].push_back(i);
    }
    dfs(1);
    getCore(1,0);
    dealRdp();
    // for(int i=1;i<=n;i++){
    //     cout<<" ** "<<core_dp[i]<<"\n";
    //     cout<<"i: "<<i<<" dep[i] "<<dep[i]<<"\n";
    //     for(int j=0;j<=log2(dep[i])+1;j++){
    //         cout<<"jump ["<<i<<"]["<<j<<"] = "<<jump[i][j]<<"\n";
    //         cout<<"rdp ["<<i<<"]["<<j<<"] = "<<r_dp[i][j]<<"\n";
    //     }
    // }cout.flush();
    for(int que=0;que<q;que++){
        int u,v;cin>>u>>v;
        int r=lca(u,v);
        // cout<<"LCA "<<u<<" "<<v<<" is : "<<r<<"\n";cout.flush();
        int ans=core_dp[r];
        if(z[r]){ans=0;}
        while(u!=r){
```

```
            for(int i=19;i>=0;i--){
                if(dep[u]-(1<<i)>=dep[r]){
                    ans+=r_dp[u][i];ans%=mod;
                    u=jump[u][i];
                    break;
                }
            }
        }
        while(v!=r){
            for(int i=19;i>=0;i--){
                if(dep[v]-(1<<i)>=dep[r]){
                    ans+=r_dp[v][i];ans%=mod;
                    v=jump[v][i];
                    break;
                }
            }
        }
        cout<<ans<<"\n";
    }
}
signed main(){
    ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);
    int t;cin>>t;
    while(t--){
        solve();
    }
}
```